

UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE CIENCIAS

MODELADO Y PROGRAMACIÓN

DOCUMENTACIÓN

PRÁCTICA 5:
Test Driven Development

DESARROLLO DE UNA FUNCIONALIDAD SIGUIENDO TDD

P R E S E N T A N :

INTEGRANTES DEL EQUIPO:

Alan Gael Alvarez Hernández
María Fernanda Miranda Sánchez
Pedro Pablo Ruiz Hernández

PROFESORA:
Karla Adriana Esquivel Guzmán

AYUDANTE DE TEORÍA:
Ossiel Aguilar Espíndola

AYUDANTE DE LABORATORIO:
Enoc Pineda Aguirre



FACULTAD DE CIENCIAS
UNAM

SEMESTRE 1-2027

Ciudad de México, 20 de octubre de 2026

Índice

1. Introducción **3**

1.1. Test Driven Development (TDD) 3

1.2. Objetivo 3

1.3. Antes de Programar 3

2. Desarrollo **4**

2.1. Implementación bajo la metodología TDD 4

2.2. Planteamiento del problema 4

2.3. Funcionalidad 1: Envío estándar 5

2.4. Funcionalidad 2: Paquete pesado 5

2.5. Funcionalidad 3: Envío exprés 5

2.6. Funcionalidad 4: Validación de peso 5

2.7. Funcionalidad 5: Tipo de envío inválido 5

2.8. Registro de Funcionalidades 5

2.9. Pruebas mínimas esperadas 5

3. Reflexión final **5**

4. Registro de Actividades **6**

Introducción

Test Driven Development (TDD)

Test-Driven Development (TDD), o desarrollo dirigido por pruebas, es una metodología de desarrollo en la que las pruebas automatizadas se utilizan como guía para construir la implementación.

El proceso se organiza en ciclos muy pequeños:

1. Escribir una prueba que describa un comportamiento deseado.
2. Ejecutar la prueba y comprobar que falla.
3. Escribir la cantidad mínima de código necesaria para hacerla pasar.
4. Ejecutar nuevamente las pruebas.
5. Refactorizar el código si es necesario.
6. Comprobar nuevamente que todas las pruebas siguen pasando.

Objetivo

Desarrollar una funcionalidad pequeña utilizando Test-Driven Development (TDD). El objetivo principal de esta práctica no es terminar rápido, sino aprender a construir el programa mediante ciclos pequeños de:

Rojo $\longrightarrow$ **Verde** $\longrightarrow$ **Refactorización** $\longrightarrow$ **Verde**

Figura 1: Flujo de trabajo con la metodología TDD.

Antes de Programar

Responder brevemente:

1. ¿Qué significa que una prueba esté en estado ROJO?

   La implementación del programa no satisface el resultado esperado por la prueba. Es decir, el comportamiento del programa observado en la prueba no es el mismo que el esperado por esta última.

2. ¿Qué debe hacerse para llegar a VERDE?

   Escribir solamente el código necesario y "útil" para que el comportamiento de la implementación sea el esperado por la prueba. Sin embargo, cabe aclarar que no se permite hacer **happy path** en las pruebas, se debe trabajar para encontrar la solución general al problema, no sólo para ciertos casos particulares del mismo.

3. ¿Qué diferencia existe entre agregar funcionalidad y refactorizar?

   Agregar funcionalidad implica generar una nueva prueba para definir un nuevo comportamiento esperado por la implementación, en cambio refactorizar se centra en simplificar el código escrito que pasó una prueba, manteniendo el comportamiento esperado por las pruebas.

4. ¿Por qué en TDD no conviene implementar todos los requisitos desde el inicio?

   De hacerlo, corremos el riesgo de que la implementación realizada no pase todas las pruebas que establezacamos en el modelo, y con ello volveremos a analizar toda la implementación sin tener conocimiento sobre donde se localiza el error a solucionar, entonces el desarrollo de código y documentción se vuelve engorroso y frustrante.

Desarrollo

Implementación bajo la metodología TDD

El flujo de trabajo consistirá en trabajar por ciclos:

$$\textbf{Rojo} \longrightarrow \textbf{Verde} \longrightarrow \textbf{Refactorización} \longrightarrow \textbf{Verde}$$

De tal forma que se agregan nuevas funcionalidades de forma incremental, hasta que terminemos de agregar todos los requisitos del modelo planteado.

Restricciones

- No implementar todos los requisitos antes de escribir las pruebas.
- No borrar pruebas únicamente para conseguir que el conjunto quede en verde.
- No modificar el resultado esperado de una prueba para adaptarlo a una implementación incorrecta.
- Cada requisito nuevo debe comenzar con al menos una prueba que falle.
- Todas las pruebas anteriores deben ejecutarse después de cada cambio.

Planteamiento del problema

Se desarrollará una función que calcule el costo de envío de un paquete.

La función recibirá, como mínimo:

- El peso del paquete en kilogramos;
- El tipo de envío.

Nota:

La funcionalidad debe construirse poco a poco. **No** implementar todos los requisitos desde el principio.

Funcionalidad 1: Envío estándar

Funcionalidad 2: Paquete pesado

Funcionalidad 3: Envío exprés

Funcionalidad 4: Validación de peso

Funcionalidad 5: Tipo de envío inválido

Registro de Funcionalidades

Ciclo	Prueba agregada	Cambio realizado	Resultado
1	-	-	-
2	-	-	-
3	-	-	-
4	-	-	-
5	-	-	-

Tabla 1: Registo de Funcionalidades Implementadas.

Pruebas mínimas esperadas

El conjunto de pruebas debe incluir, al menos:

- ☐ Envío estándar menor a 5 kg.
- ☐ Envío estándar exactamente de 5 kg.
- ☐ Envío estándar mayor a 5 kg.
- ☐ Envío exprés menor a 5 kg.
- ☐ Envío exprés mayor a 5 kg.
- ☐ Peso igual a 0 kg.
- ☐ Peso negativo.
- ☐ Tipo de envío inválido.

Reflexión final

Máximo 5 líneas.

Responder:

1. ¿En qué momento una prueba te obligó a modificar el diseño?

1. ¿Qué ventaja encontraste al ejecutar todas las pruebas después de cada cambio?

1. ¿Qué parte del proceso TDD te resultó más difícil?

Registro de Actividades

Actividades pendientes:

- ⊟ Documentación.
- ☐ Funcionalidad 1.
- ☐ Funcionalidad 2.
- ☐ Funcionalidad 3.
- ☐ Funcionalidad 4.
- ☐ Funcionalidad 5.
- ☐ Registro de Funcionalidades.
- ☐ Reflexión final.
- ⊟ Organización de Archivos.
- ⊟ README.md.